# 11 · Services and Dependency Injection

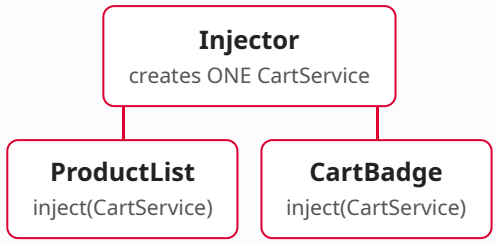
Intermediate Angular learning notes

## a. Definition

A **service** is a plain class that holds logic or data you want to share (loading data, a shopping cart, logging). Components should focus on the screen; services do the real work.

**Dependency Injection (DI)** means Angular creates the service for you and hands it over when a class asks for it with `inject(ServiceName)`. With `providedIn: 'root'` there is exactly **one shared instance** for the whole app, so every component sees the same data.

## b. Diagram




Both components receive the same service object, so they stay in sync.

## c. Syntax

```
@Injectable({ providedIn: 'root' })
export class DataService {
  getItems() { return ['a', 'b']; }
}

@Component({...})
export class MyComponent {
  private data = inject(DataService);   // ask Angular for it
}
```

## d. Example Program

Programs/11_Services_and_DI/cart-service.component.ts

```
import { Component, computed, inject, Injectable, signal } from '@angular/core';

// A service: ONE shared instance for the whole app (providedIn: 'root')
@Injectable({ providedIn: 'root' })
export class CartService {
  private readonly items = signal<string[]>([]);

  readonly list = this.items.asReadonly(); // others can read, but not change directly
  readonly count = computed(() => this.items().length);

  add(item: string) {
    this.items.update((current) => [...current, item]);
  }
```

```
  clear() {
    this.items.set([]);
  }
}

// Component 1 writes to the cart
@Component({
  selector: 'app-product-list',
  template: `
    @for (p of products; track p) {
      <button (click)="cart.add(p)">Add {{ p }}</button>
    }
  `,
})
export class ProductListComponent {
  products = ['Idli Batter', 'Filter Coffee Powder'];
  cart = inject(CartService); // dependency injection
}

// Component 2 reads the SAME cart
@Component({
  selector: 'app-cart-badge',
  template: `
    <p>Cart items: {{ cart.count() }}</p>
    <ul>
      @for (item of cart.list(); track $index) { <li>{{ item }}</li> }
    </ul>
  `,
})
export class CartBadgeComponent {
  cart = inject(CartService);
}

@Component({
  selector: 'app-root',
  imports: [ProductListComponent, CartBadgeComponent],
  template: `
    <h2>Online Grocery</h2>
    <app-product-list />
    <app-cart-badge />
  `,
})
export class AppComponent {}
```

## e. Output

Rendered in the browser (text on screen, after the UI steps below):

1. Click **Add Idli Batter** twice.
2. Click **Add Filter Coffee Powder** once.

```
Online Grocery
Add Idli BatterAdd Filter Coffee Powder
Cart items: 3
Idli Batter
Idli Batter
Filter Coffee Powder
```

## f. Real-World Example (English + தமிழ்)

**English**

A whole village shares one public water tap. A service is that single shared tap; dependency injection is the municipality bringing the connection to your home.

**தமிழ்**

ஒரு ஊரில் எல்லாரும் ஒரே பொதுக் குடிநீர்க் குழாயைப் பயன்படுத்துகிறார்கள். ஒவ்வொரு வீடும் தனியாக ஆழ்குழாய்க் கிணறு தோண்டத் தேவையில்லை. ஒரு வீட்டில் தண்ணீர் எடுத்தால் மற்ற வீடுகளுக்கும் அதே நீர்த்தொட்டியின் நிலை தெரியும்.

**Service** அந்தப் பொதுக் குழாய் போல - ஒரே ஒரு பிரதி, எல்லா component-களும் பகிர்ந்துகொள்ளும். **Dependency Injection** என்பது குழாய் இணைப்பை உங்கள் வீடு வரை கொண்டுவந்து கொடுக்கும் நகராட்சி ஊழியர் போல; நாமே கிணறு தோண்ட வேண்டியதில்லை, `inject()` என்று கேட்டால் போதும்.

## g. Pros

- Shared logic and state
- Easy to mock in tests
- Loose coupling

## h. Cons

- Singletons can hide global state
- Wrong provider scope causes confusing bugs

## i. Advanced / New Features

- `inject()` function (replaces constructor injection)
- `provideAppInitializer`
- Tree-shakable `providedIn: 'root'`

## j. Common Mistakes

- Creating a service with `new CartService()` instead of injecting it; you lose the shared instance.
- Forgetting `@Injectable({ providedIn: 'root' })`, which gives a "No provider" error.
- Putting HTTP calls and business logic inside components instead of services.
- Exposing a writable signal publicly so any component can change it unexpectedly.

## k. Interview Questions

**Q1. What is a service in Angular?**
A class that encapsulates reusable logic or state, injected into components and other services.

**Q2. What is dependency injection?**
A design pattern where a class receives its dependencies from an injector instead of creating them itself.

**Q3. What does `providedIn: 'root'` do?**
Registers the service in the root injector, creating a single app-wide instance (tree-shakable).

**Q4. `inject()` vs constructor injection?**
Both work. `inject()` is shorter, works in field initialisers and functions (guards, interceptors), and is the modern style.

## l. Practice Exercises

1. Create a `ThemeService` that stores 'light' or 'dark' and use it in two components.
2. Make a `LoggerService` with `log(message)` and call it from three components.
3. Add a `remove(item)` method to `CartService` and a remove button next to each item.